

Computer Architecture

(EE 371 / CE 321 / CS 330)

Dr. Farhan Khan
Assistant Professor,
Dhanani School of Science & Engineering,
Habib University

Outline: Unit 2

- Introduction to Instruction Sets
- RISC-V Instruction Set
- Instruction Encoding

Instruction Set

- Instruction Set
 - The vocabulary of instructions a computer can understand
- Different Computers have different instruction sets
 - But with many common aspects
- Examples
 - RISC-V
 - MIPS
 - Intel x86

Let's Wear the Hat of an Instruction Set Designer

- Goals of an instruction set designer

Let's Wear the Hat of an Instruction Set Designer

- Goals of an instruction set designer
 - Make it easy to build the hardware

Let's Wear the Hat of an Instruction Set Designer

- Goals of an instruction set designer
 - Make it easy to build the hardware
 - Make it easy to build the compiler

Let's Wear the Hat of an Instruction Set Designer

- Goals of an instruction set designer
 - Make it easy to build the hardware
 - Make it easy to build the compiler
 - Maximize performance
 - Minimize cost
 - Minimize energy consumption

Let's Wear the Hat of a Computer Systems Designer

- Choose a processor (and its associated instruction set) that offers an appropriate combination of
 - performance
 - cost
 - energy consumption

RISC-V Instruction Set

- Used as the example throughout the book
- Developed at UC Berkeley as open ISA
- Now managed by the RISC-V Foundation (riscv.org)
- Typical of many modern ISAs
- Similar ISAs have a large share of embedded core market
 - Applications in consumer electronics, network/storage equipment, cameras, printers, ...

Categories of RISC-V Instructions

- Arithmetic
- Data Transfer
- Logical
- Shift
- Conditional Branch
- Unconditional Branch

Categories of RISC-V Instructions

- **Arithmetic**
- Data Transfer
- Logical
- Shift
- Conditional Branch
- Unconditional Branch

Arithmetic Instructions of RISC-V

Category	Instruction	Example	Meaning	Comments
Arithmetic	Add	<code>add x5, x6, x7</code>	$x5 = x6 + x7$	Three register operands; add
	Subtract	<code>sub x5, x6, x7</code>	$x5 = x6 - x7$	Three register operands; subtract
	Add immediate	<code>addi x5, x6, 20</code>	$x5 = x6 + 20$	Used to add constants

Arithmetic Instructions of RISC-V

- Each RISC-V arithmetic instruction must have exactly three operands
 - Two sources and one destination
 - add a, b, c // a gets $b + c$
 - sub a, b, c // a gets $b - c$

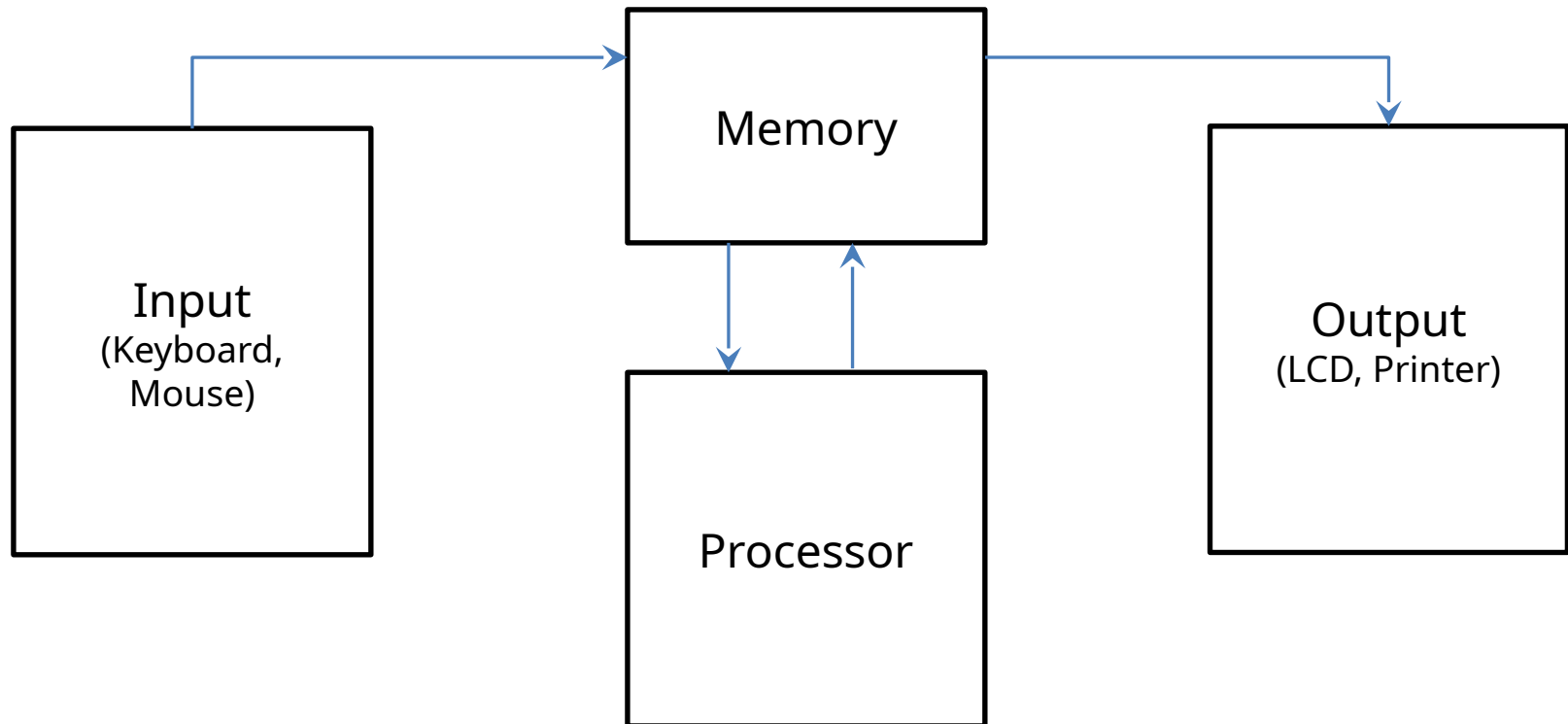
Arithmetic Instructions of RISC-V

- Each RISC-V arithmetic instruction must have exactly three operands
 - Two sources and one destination
 - add a, b, c // a gets $b + c$
 - sub a, b, c // a gets $b - c$
- What are operands a, b, c?

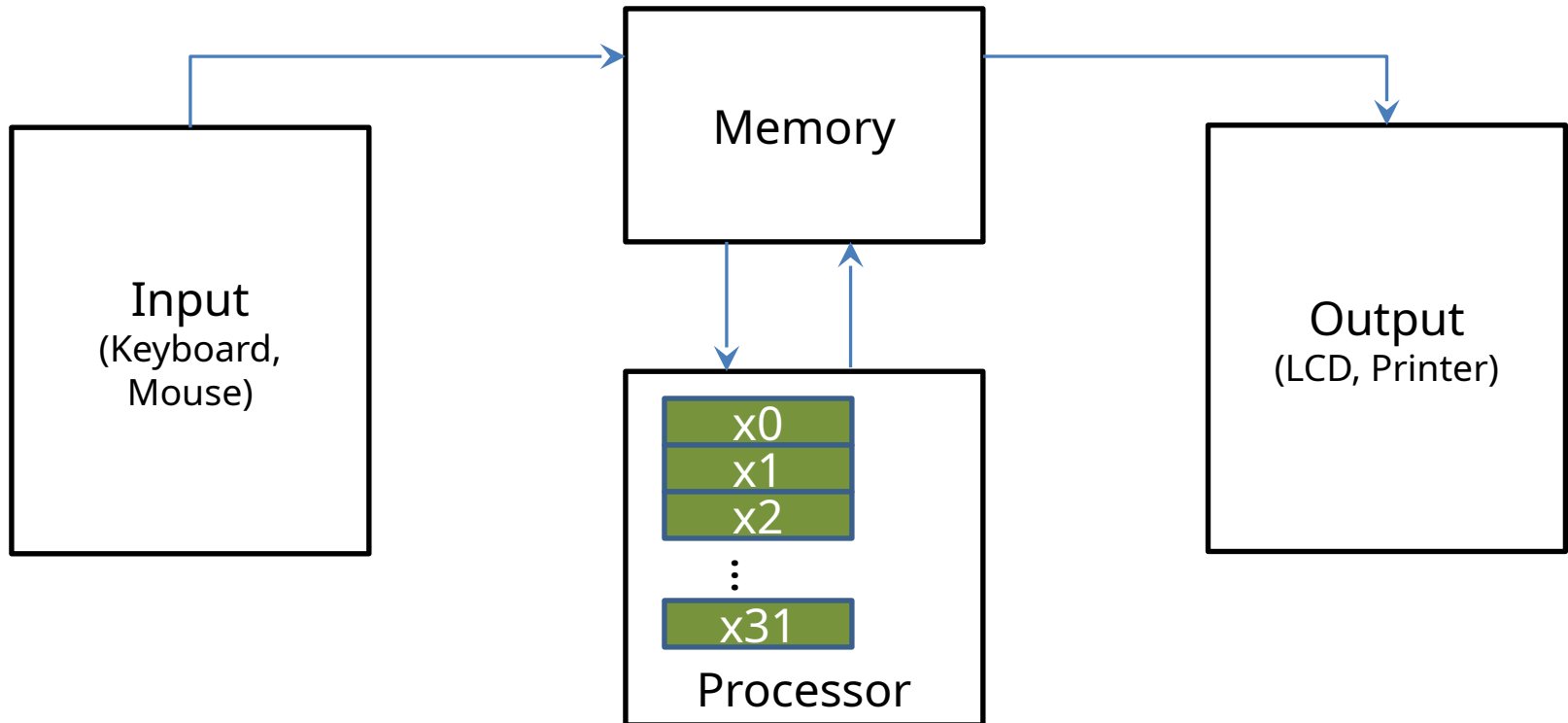
Register Operands

- Arithmetic instructions use register operands
 - Two sources and one destination
 - `add x5, x20, x21` // x5 gets $x20 + x21$
 - `sub x5, x20, x21` // x5 gets $x20 - x21$

Computer Architecture: The Big Picture



Computer Architecture: The Big Picture



Register Operands

- Arithmetic instructions use register operands
 - Two sources and one destination
 - `add x5, x20, x21` // x5 gets $x20 + x21$
 - `sub x5, x20, x21` // x5 gets $x20 - x21$
- RISC-V has 32 registers
- Each register is a 64-bit register

RISC-V Registers

- x0: the constant value 0
- x1: return address
- x2: stack pointer
- x3: global pointer
- x4: thread pointer
- x5 – x7, x28 – x31: temporaries
- x8: frame pointer
- x9, x18 – x27: saved registers
- x10 – x11: function arguments/results
- x12 – x17: function arguments

Register Operands

- Arithmetic instructions use register operands
 - Two sources and one destination
 - `add x5, x20, x21` // x5 gets $x20 + x21$
 - `sub x5, x20, x21` // x5 gets $x20 - x21$
- RISC-V has 32 registers
- Each register is a 64-bit register

Register Operand Example

- C code:
 - $f = (g + h) - (i + j);$
- Register Assignment:
 - $f, g, \dots, j$ in $x19, x20, \dots, x23$
- Compiled RISC-V assembly code:
 - `add x5, x20, x21`
`add x6, x22, x23`
`sub x19, x5, x6`

Immediate Operands

- Arithmetic instructions can use immediate/constant operands
 - `addi x5, x20, 4` // x5 gets x20 + 4
- Constant operands occur frequently; indeed, `addi` is the most popular instruction in most RISC-V programs.
- RISC-V dedicates register `x0` to be hard-wired to the value zero.

Design Insights

- Goals of an instruction set designer
 - Make it easy to build the hardware
 - Make it easy to build the compiler
 - Maximize performance
 - Minimize cost
 - Minimize energy consumption
- Requiring every instruction to have exactly 3 operands, no more no less, conforms to the philosophy of keeping the hardware simple
 - Hardware for variable number of operands is more complicated than hardware for fixed number of operands

Categories of RISC-V Instructions

- Arithmetic
- **Data Transfer**
- Logical
- Shift
- Conditional Branch
- Unconditional Branch

Need for Data Transfer Instructions

- Programming languages have simple variables that contain single data elements, but they also have more complex data structures—arrays and structures.
- These composite data structures can contain many more data elements than there are registers in a computer.
- These data structure (arrays and structures) are kept in memory
- Arithmetic operations occur only on registers in RISC-V instructions
- As a result, RISC-V must include instructions that transfer data between memory and registers.

Data Transfer Instructions

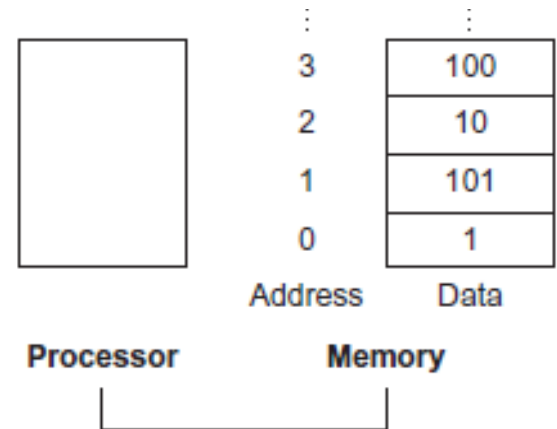
Category	Instruction	Example	Meaning	Comments
Data transfer	Load doubleword	ld x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Doubleword from memory to register
	Store doubleword	sd x5, 40(x6)	$\text{Memory}[x6 + 40] = x5$	Doubleword from register to memory
	Load word	lw x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Word from memory to register
	Load word, unsigned	lwu x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Unsigned word from memory to register
	Store word	sw x5, 40(x6)	$\text{Memory}[x6 + 40] = x5$	Word from register to memory
	Load halfword	lh x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Halfword from memory to register
	Load halfword, unsigned	lhu x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Unsigned halfword from memory to register
	Store halfword	sh x5, 40(x6)	$\text{Memory}[x6 + 40] = x5$	Halfword from register to memory
	Load byte	lb x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Byte from memory to register
	Load byte, unsigned	lbu x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	Byte unsigned from memory to register
	Store byte	sb x5, 40(x6)	$\text{Memory}[x6 + 40] = x5$	Byte from register to memory
	Load reserved	lr.d x5, (x6)	$x5 = \text{Memory}[x6]$	Load; 1st half of atomic swap
	Store conditional	sc.d x7, x5, (x6)	$\text{Memory}[x6] = x5; x7 = 0/1$	Store; 2nd half of atomic swap
	Load upper immediate	lui x5, 0x12345	$x5 = 0x12345000$	Loads 20-bit constant shifted left 12 bits

Data Transfer Instructions

- Load Instructions
 - Load values from memory into registers
- Store Instructions
 - Store values from registers into memory

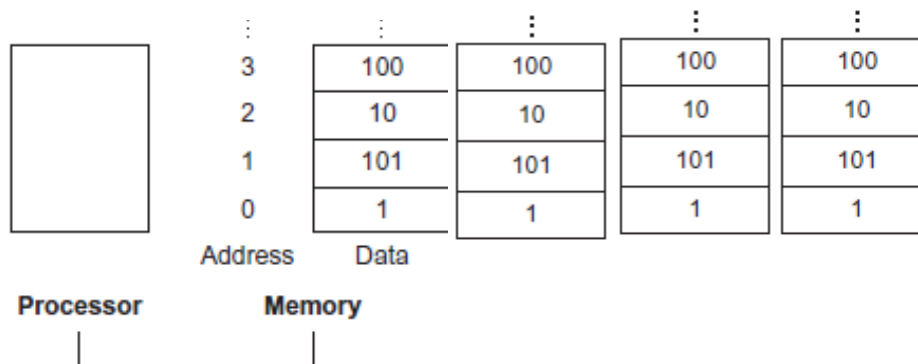
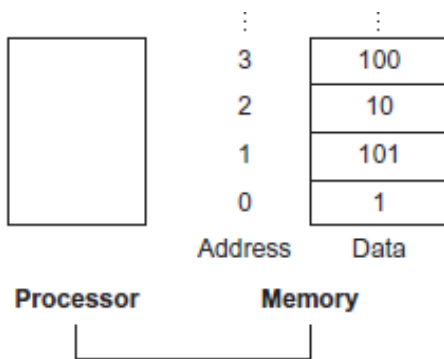
Memory Addressing

- To access a value in memory, the instruction must provide an address
- Load Instructions
 - Load register, memory address
- Store Instructions
 - Store register, memory address



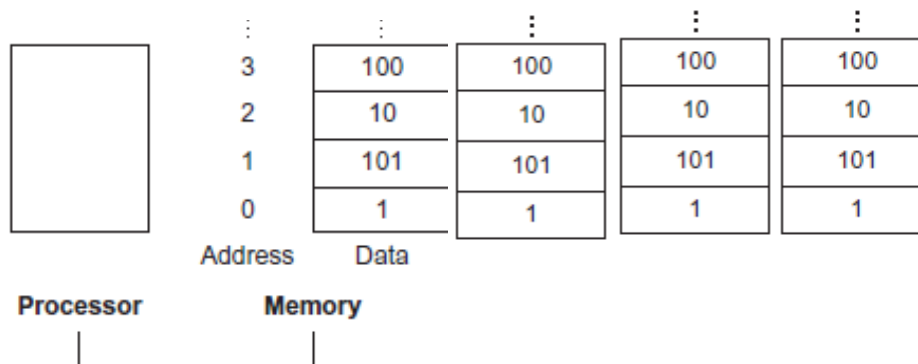
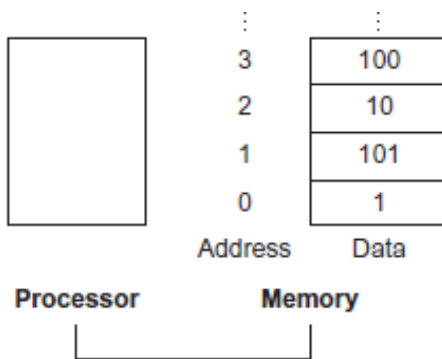
Memory Addressing

- How many bytes does each address identify



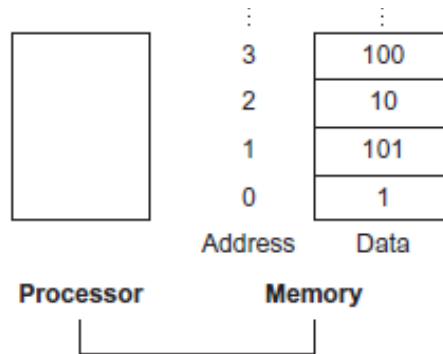
Memory Addressing

- How many bytes does each address identify
 - In RISC-V, each memory address identifies one byte (8 bits)



Memory Addressing

- How many bytes does each address identify
 - In RISC-V, each memory address identifies one byte (8 bits)
 - Load register, memory address
 - Store register, memory address



- LoadByte from 0, LoadByte from 1, LoadByte from 2,...
- LoadWord from 0, LoadWord from 4, LoadWord from 8,
- LoadDoubleWord from 0, LoadDoubleWord from 8, LoadDoubleWord from 16,

Memory Addressing

- Memory address is described through a **base register** and **offset**
 - Load register, memoryaddress
 - **Load register, offset(baseRegister)**
- Consider
 - char cArray[10];
 - int iArray[10];
 - Long lArray[10];

Memory Addressing

- Consider
char cArray[10];

Memory Addressing

- Consider
 `int iArray[10];`

Memory Addressing

- Consider
 `long lArray[10];`

Memory Addressing

- Consider
 long lArray[10];

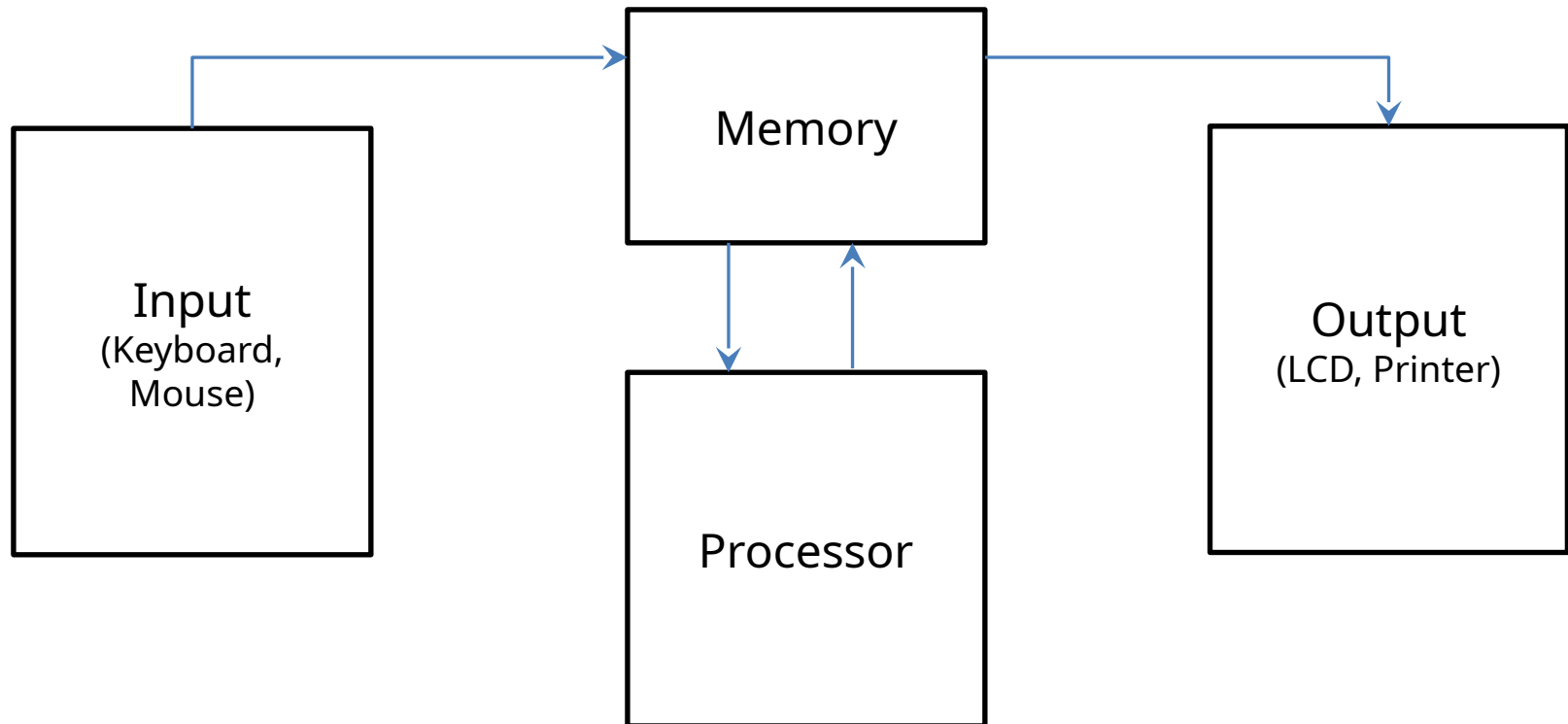
Example

- C code:
 - $A[12] = h + A[8];$
- h in $x21$, base address of A in $x22$
- Compiled RISC-V assembly code:
 - | | | |
|-----|-------------|-------------|
| ld | | x9, 64(x22) |
| add | x9, x21, x9 | |
| sd | x9, 96(x22) | |

Design Insights

- Registers are faster to access than memory
- Operating on memory data requires loads and stores
 - More instructions to be executed
- Compiler must use registers for variables as much as possible
 - Only spill to memory for less frequently used variables
 - Register optimization is important!

Computer Architecture: The Big Picture



Design Insights

- If arithmetic instructions with immediate operands was not part of Instruction Set?
 - Utilize an extra register
 - Loading a value from memory
 - Performance and energy consumption hit

RISC-V: Loading a Byte from Memory into a Register

Load byte	<code>lb x5, 40(x6)</code>	<code>x5 = Memory[x6 + 40]</code>	Byte from memory to register
Load byte, unsigned	<code>lbui x5, 40(x6)</code>	<code>x5 = Memory[x6 + 40]</code>	Byte unsigned from memory to register

- `lb`:
 - Sign extend loaded byte
- `lbui`:
 - Zero extend loaded byte

Unsigned Binary Integers

- Given an n-bit number

$$x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \dots + x_12^1 + x_02^0$$

- Range: 0 to $+2^n - 1$

- Example

$$\begin{aligned} & - 0000\ 0000 \dots 0000\ 1011_2 \\ & = 0 + \dots + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ & = 0 + \dots + 8 + 0 + 2 + 1 = 11_{10} \end{aligned}$$

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48	47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

Signed Binary Integers: 2's Complement Representation

- Given an n-bit number

$$X = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \dots + x_12^1 + x_02^0$$

- Range: -2^{n-1} to $+2^{n-1} - 1$
- Example
 - $1111\ 1111\ \dots\ 1111\ 1100_2$
 $= -1 \times 2^{31} + 1 \times 2^{30} + \dots + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$
 $= -2,147,483,648 + 2,147,483,644 = -4_{10}$

2's Complement Representation: Signed Negation

- Complement and add 1
 - Complement means $1 \rightarrow 0, 0 \rightarrow 1$
- Example: negate +2
 - $+2 = 0000\ 0000 \dots 0010_{\text{two}}$
 - $-2 = 1111\ 1111 \dots 1101_{\text{two}} + 1$
 $= 1111\ 1111 \dots 1110_{\text{two}}$

2's Complement Representation

- Most significant bit is sign bit
 - 1 for negative numbers
 - 0 for non-negative numbers
- Non-negative numbers have the same unsigned and 2s-complement representation
- Some specific numbers
 - 0: 0000 0000 ... 0000
 - -1: 1111 1111 ... 1111
 - Most-negative: 1000 0000 ... 0000
 - Most-positive: 0111 1111 ... 1111

$$X = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \dots + x_12^1 + x_02^0$$

RISC-V: Loading a Byte from Memory into a Register

Signed or Unsigned Interpretation

- Given the following 16-bits in memory

0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Signed Interpretation: +2
Interpretation: 2

Unsigned

- Given the following 16 bits in memory

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Signed Interpretation: -2
65534

Unsigned Interpretation:

Signed or Unsigned Interpretation

- Signed and Unsigned Data Types in C (a high-level language)

int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	8 bytes or (4bytes for 32 bit OS)	-9223372036854775808 to 9223372036854775807
unsigned long	8 bytes	0 to 18446744073709551615

Signed or Unsigned Interpretation

- Given the following 16-bits in memory

0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Signed Interpretation: +2
Interpretation: 2

short var1 = +2;

Unsigned

unsigned short var1 = 2;

- Given the following 16 bits in memory

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Signed Interpretation: -2
65,534

short var1 = -2;
65534;

Unsigned Interpretation:

unsigned short var1 =

Representing a Number using More bits while Preserving the Numeric Value

- For Signed Numbers:
 - Replicate the sign bit to the left (Sign Extension)
- For Unsigned Numbers:
 - Extend with 0s (Zero Extension)
- Examples: 8-bit to 16-bit
 - +2: 0000 0010 => 0000 0000 0000 0010
 - -2: 1111 1110 => 1111 1111 1111 1110

RISC-V: Loading a Byte from Memory into a Register

Load byte	<code>lb x5, 40(x6)</code>	<code>x5 = Memory[x6 + 40]</code>	Byte from memory to register
Load byte, unsigned	<code>lbui x5, 40(x6)</code>	<code>x5 = Memory[x6 + 40]</code>	Byte unsigned from memory to register

- `lb`:
 - Sign extend loaded byte
- `lbui`:
 - Zero extend loaded byte

RISC-V: Loading a Halfword from Memory into a Register

Load halfword	<code>lh x5, 40(x6)</code>	<code>x5 ← Memory[x6 + 40]</code>	Halfword from memory to register
Load halfword, unsigned	<code>lhu x5, 40(x6)</code>	<code>x5 ← Memory[x6 + 40]</code>	Unsigned halfword from memory to register

- `lh`:
 - Sign extend loaded halfword
- `lhu`:
 - Zero extend loaded halfword

Categories of RISC-V Instructions

- Arithmetic
- Data Transfer
- **Logical**
- **Shift**
- Conditional Branch
- Unconditional Branch

RISC-V: Shift and Logical Instructions

Logical operations	C operators	Java operators	RISC-V instructions
Shift left	<<	<<	sll, slli
Shift right	>>	>>>	srl, srli
Shift right arithmetic	>>	>>	sra, srai
Bit-by-bit AND	&	&	and, andi
Bit-by-bit OR			or, ori
Bit-by-bit XOR	^	^	xor, xori
Bit-by-bit NOT	~	~	xori

Shift Instructions

- Shift Left
 - Shift left and fill the emptied bits with 0
 - Shift left logical (sll): sll x11, x19, x20
 - Shift left logical immediate (slli): slli x11, x19, 4
- Shift Right
 - Shift right and fill the emptied bits with 0
 - Shift right logical (srl): srl x11, x19, x20
 - Shift right logical immediate (srli): srli x11, x19, 4
- Shift Right Arithmetic
 - Shift right and fill the emptied bits with copies of old sign bit
 - Shift right arithmetic (sra): sra x11, x19, x20
 - Shift right arithmetic immediate (srai): srai x11, x19, 4

Shift Left

- Shift left and fill the emptied bits with 0
- Shift left logical (sll): sll x11, x19, x20
- Shift left logical immediate (slli): slli x11, x19, 4
- Shifting left by i bits gives the same result as multiplying by 2^i

Shift Right

- Shift right and fill the emptied bits with 0
- Shift right logical (srl): srl x11, x19, x20
- Shift right logical immediate (srli): srli x11, x19, 4

Shift Right Arithmetic

- Shift right and fill the emptied bits with copies of old sign bit
- Shift right arithmetic (sra): sra x11, x19, x20
- Shift right arithmetic immediate (srai): srai x11, x19, 4

AND Operation

- A bit-by-bit operation that leaves a 1 in the result only if both bits of the operands are 1.

and x9,x10,x11

x10	00000000 00000000 00000000 00000000 00000000 00000000 00001101 11000000
x11	00000000 00000000 00000000 00000000 00000000 00000000 00111100 00000000
x9	00000000 00000000 00000000 00000000 00000000 00000000 00001100 00000000

OR Operation

- A bit-by-bit operation that places a 1 in the result if either operand bit is a 1
or x9,x10,x11

x10	00000000 00000000 00000000 00000000 00000000 00000000 00001101 11000000
x11	00000000 00000000 00000000 00000000 00000000 00000000 00111100 00000000
x9	00000000 00000000 00000000 00000000 00000000 00000000 00111101 11000000

XOR Operation

- A bit-by-bit operation that creates a 0 in the result when bits in the operand are the same and a 1 if they are different.

xor x9,x10,x11

x10	00000000	00000000	00000000	00000000	00000000	00000000	00001101	11000000
x11	11111111	11111111	11111111	11111111	11111111	11111111	11111111	11111111
x9	11111111	11111111	11111111	11111111	11111111	11111111	11110010	00111111

NOT Operation

- In RISC-V, NOT operation is implemented through XOR
- NOT is equivalent to an XOR with 1111...1111

xor x9,x10,x11

x10	00000000	00000000	00000000	00000000	00000000	00000000	00001101	11000000
x11	11111111	11111111	11111111	11111111	11111111	11111111	11111111	11111111
x9	11111111	11111111	11111111	11111111	11111111	11111111	11110010	00111111

Categories of RISC-V Instructions

- Arithmetic
- Data Transfer
- Logical
- Shift
- **Conditional Branch**
- **Unconditional Branch**

Conditional Branches

- Branch to a labeled instruction if a condition is true
 - Otherwise, continue sequentially
- **beq rs1, rs2, L1**
 - if (rs1 == rs2) branch to instruction labeled L1
- **bne rs1, rs2, L1**
 - if (rs1 != rs2) branch to instruction labeled L1

Conditional Branches: Compiling If Statements

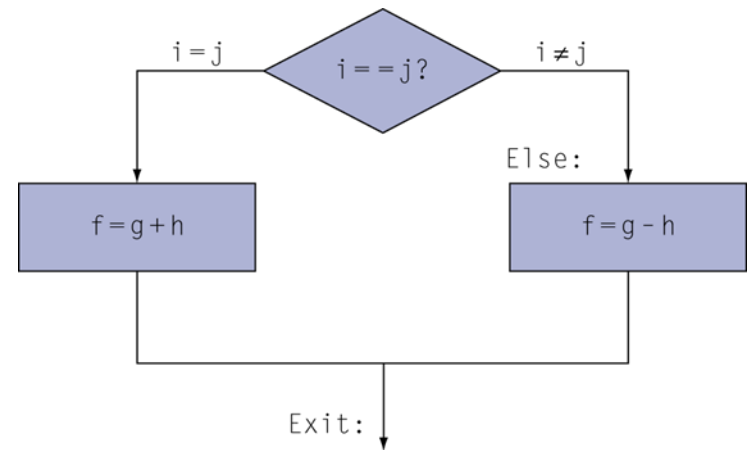
- C code:

- if (i==j) f = g+h;
else f = g-h;

- f, g, ... in x19, x20, ...

- Compiled RISC-V code:

- bne x22, x23, Else
add x19, x20, x21
beq x0,x0,Exit // unconditional
Else: sub x19, x20, x21
Exit: ...



Conditional Branches: Compiling If Statements

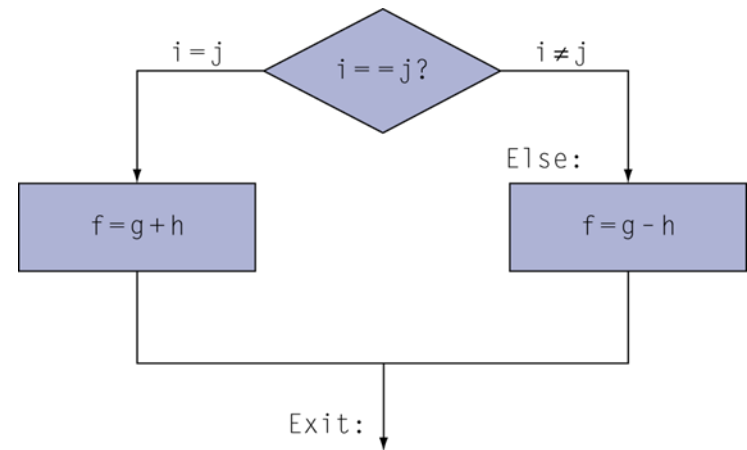
- C code:

- if (i==j) f = g+h;
else f = g-h;

- f, g, ... in x19, x20, ...

- Compiled RISC-V code:

- bne x22, x23, Else
add x19, x20, x21
beq x0,x0,Exit // unconditional
Else: sub x19, x20, x21
Exit: ...



Assembler calculates addresses

Conditional Branches: Compiling Loop Statements

- C code:
 - while (save[i] == k)
 i += 1;
- i in x22, k in x24, address of save in x25
- Compiled RISC-V code:
 - Loop: slli x10, x22, 3
 add x10, x10, x25
 ld x9, 0(x10)
 bne x9, x24, Exit
 addi x22, x22, 1
 beq x0, x0, Loop
Exit: ...

More Conditional Branches

- **blt rs1, rs2, L1**
 - if ($rs1 < rs2$), branch to instruction labeled L1
- **bge rs1, rs2, L1**
 - if ($rs1 \geq rs2$), branch to instruction labeled L1
- **bltu rs1, rs2, L1**
 - rs1, rs2 are treated as unsigned numbers
 - if ($rs1 < rs2$), branch to instruction labeled L1
- **bgeu rs1, rs2, L1**
 - rs1, rs2 are treated as unsigned numbers
 - if ($rs1 \geq rs2$), branch to instruction labeled L1

More Conditional Branches: Example

- C code:
 - if (a > b)
 a += 1;
- a in x22, b in x23
- Compiled RISC-V code:
 - bge x23, x22, Exit // branch if b >= a
 addi x22, x22, 1
Exit:

More Conditional Branches: Elaborating Signed vs Unsigned

- Signed comparison: blt, bge
- Unsigned comparison: bltu, bgeu
- Example
 - $x_{22} = 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111$
 - $x_{23} = 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001$
 - $x_{22} < x_{23}$ // signed
 $-1 < +1$
 - $x_{22} > x_{23}$ // unsigned
 $4,294,967,295 > 1$

RISC-V: Procedure Calling

- Steps required
 - Place parameters in registers x10 to x17
 - Transfer control to procedure and store return address in x1
 - Acquire storage for procedure
 - Perform procedure's operations
 - Place result in register for caller
 - Return to place of call (address in x1)

RISC-V: Procedure Call Instructions

- Procedure call: jump and link
 - **jal x1, ProcedureLabel**
 - Address of following instruction put in x1
 - Jumps to target address
- Procedure return: jump and link register
 - **jalr x0, 0(x1)**
 - Like jal, but jumps to 0 + address in x1
 - Use x0 as rd (x0 cannot be changed)

RISC-V: Procedure Calling (Summary v1)

- *The Caller (The calling procedure)*
 - *puts the parameter values in x10-x17 and*
 - *uses jal x1, X to branch to procedure X (sometimes named the callee).*
- *The Callee (The called procedure)*
 - *performs the calculations,*
 - *places the results in the same parameter registers (x10-x17)*
 - *returns control to the caller using jalr x0, 0(x1).*

RISC-V Procedure Calling: Introducing Stack

- C code:

```
long long int leaf_example (long long int
g,
                                long long int
h,
                                long long
int i,
                                long long int
j) {
    long long int f;
    f = (g + h) - (i + j);
    return f;
}
```

- RISC-V code

- leaf_example:

	add x5,x10,x11	// x5 = g +
	add x6,x12,x13	// x6 = i + j
	sub x20,x5,x6	// f = x5 -
h		
x6		

- Assignment

- Arguments g, ..., j in x10, ..., x13
 - f in x20, g+h and i+j in x5, x6

jalr x0,0(x1)

RISC-V Procedure Calling: Introducing Stack

- C code:

```
long long int leaf_example (long long int
g,
                                long long int
h,
                                long long
int i,
                                long long int
j) {
    long long int f;
    f = (g + h) - (i + j);
    return f;
}
```

- RISC-V code

- leaf_example:

```
add x5,x10,x11    // x5 = g +
add x6,x12,x13    // x6 = i + j
sub x20,x5,x6     // f = x5 -
x6
```

- Assignment

- Arg
- f in

• What if x5, x6, x20 were being used by the Calling procedure before calling leaf_example() ?

RISC-V Procedure Calling: Introducing Stack

- C code:

```
long long int leaf_example (long long int
g,
                                long long int
h,
                                long long
int i,
                                long long int
j) {
    long long int f;
```

1. At the start of leaf_example (), save current values of x5, x6, x20 to memory

2. Utilize x5, x6, x20 in leaf_example()

3. At the end of leaf_example(), restore x5, x6, x20 to their original values stored in the memory at the start of leaf_example()

- RISC-V code

- leaf_example:

```
add x5,x10,x11    // x5 = g +
add x6,x12,x13    // x6 = i + j
sub x20,x5,x6     // f = x5 -
```

```
jalr x0,0(x1)
```

RISC-V Procedure Calling: Stack

- C code:

```
long long int leaf_example (long long int  
g,
```

The ideal data structure to save and restore x5, x6, x20 to memory is a **Stack**: last-in, first-out queue

```
int i
```

```
j) {
```

```
long long int f;
```

1. At the start of leaf_example (), save current values of x5, x6, x20 to memory

- 2. Utilize x5, x6, x20 in leaf_example()

3. At the end of leaf_example(), restore x5, x6, x20 to their original values stored in the memory at the start of leaf_example()

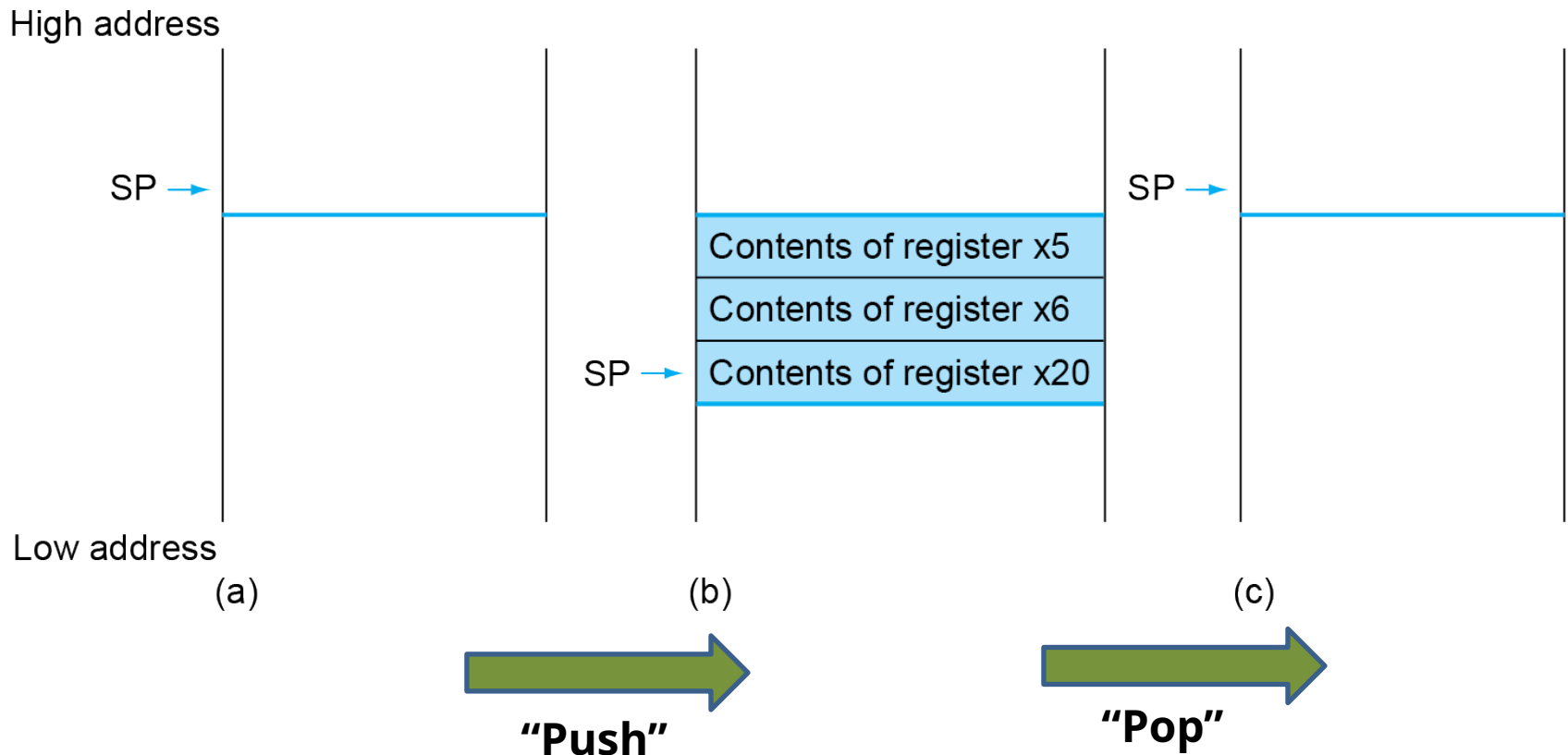
- RISC-V code

- leaf_example:

```
add x5,x10,x11    // x5 = g +  
add x6,x12,x13    // x6 = i + j  
sub x20,x5,x6     // f = x5 -
```

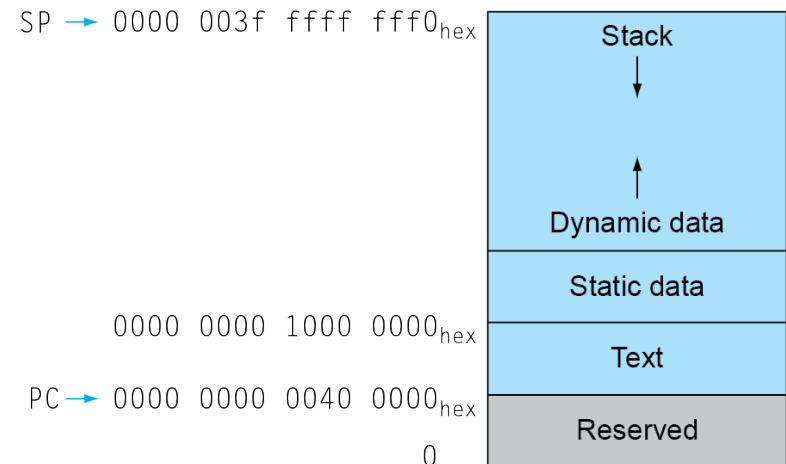
```
jalr x0,0(x1)
```

RISC-V Procedure Calling: Utilizing Stack

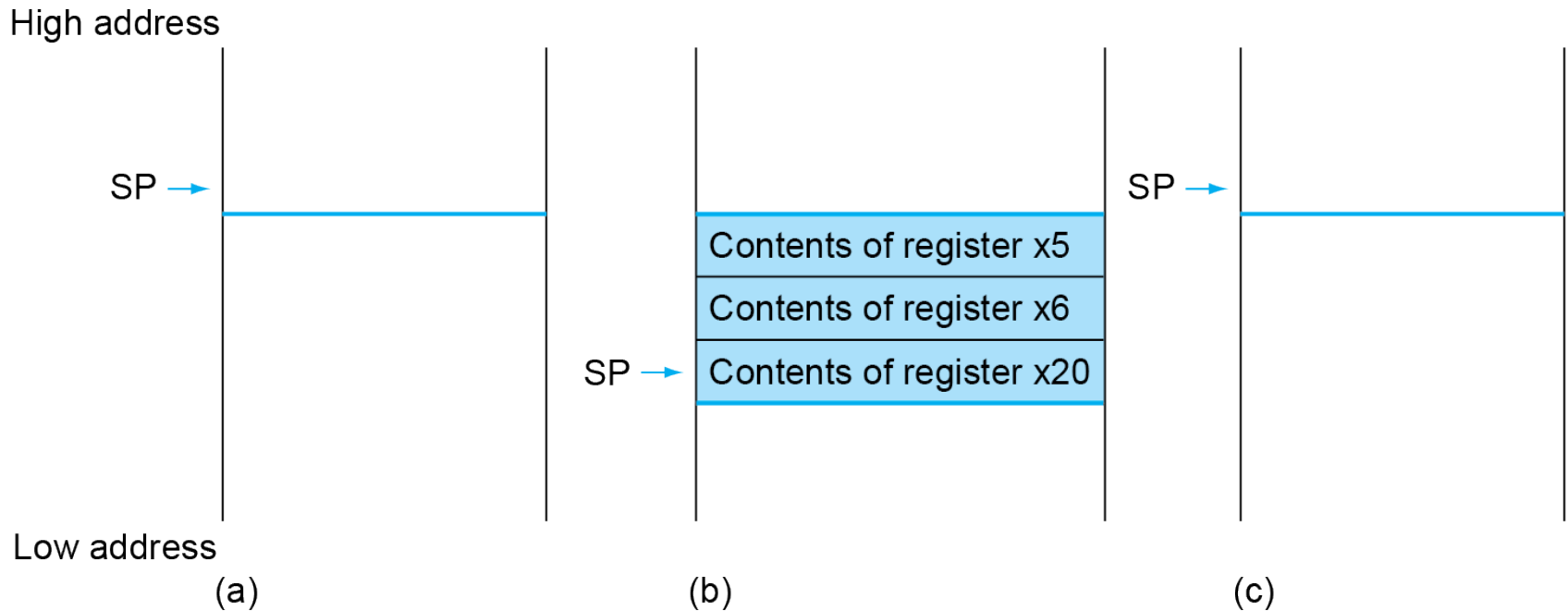


RISC-V: Memory Layout Convention

- Text
 - program machine code
- Static data
 - global variables e.g., static variables in C, constant arrays and strings
 - x3 (global pointer) initialized to the first address of this segment, allowing $\pm$ offsets into this segment
- Dynamic data (Heap)
 - Data structures like linked list that grow and shrink during their lifetime
 - malloc in C, new in Java
- Stack
 - automatic variables



RISC-V Procedure Calling: Utilizing Stack



- By historical precedent, stacks “grow” from higher addresses to lower addresses.
- This convention means that you push values onto the stack by subtracting from the stack pointer. Adding to the stack pointer shrinks the stack, thereby popping values off the stack.

RISC-V Procedure Calling: Introducing Stack

- C code:

```
long long int leaf_example (long long int
g,
                                long long int
h,
                                long long
int i,
                                long long int
j) {
    long long int f;
    f = (g + h) - (i + j);
    return f;
}
```

- RISC-V code

- leaf_example:

	add x5,x10,x11	// x5 = g +
	add x6,x12,x13	// x6 = i + j
	sub x20,x5,x6	// f = x5 -
h		
x6		

- Assignment

- Arguments g, ..., j in x10, ..., x13
 - f in x20, g+h and i+j in x5, x6
 - Need to save x5, x6, x20 on stack

jalr x0,0(x1)

RISC-V Procedure Calling: Utilizing Stack

- C code:

```
long long int leaf_example (long long int
g,
                                long long int
h,
                                long long
int i,
                                long long int
j) {
    long long int f;
    f = (g + h) - (i + j);
    return f;
}
```

- Assignment

- Arguments g, ..., j in x10, ..., x13
- f in x20, g+h and i+j in x5, x6
- Need to save x5, x6, x20 on stack

- RISC-V code

- leaf_example:

```
addi sp,sp,-24
sd x5,16(sp)
sd x6,8(sp)
sd x20,0(sp)
add x5,x10,x11    // x5 = g +
h
add x6,x12,x1      // x6 = i + j
sub x20,x5,x6      // f = x5 -
x6
addi x10,x20,0
ld x20,0(sp)
ld x6,8(sp)
ld x5,16(sp)
addi sp,sp,24
jalr x0,0(x1)
```

RISC-V Procedure Calling: Introducing Stack

- C code:

```
long long int leaf_example (long long int
g,
                                long long int
h,
                                long long
int i,
                                long long int
j) {
    long long int f;
    f = (g + h) - (i + j);
    return f;
}
```

- RISC-V code

- leaf_example:

```
add x5,x10,x11    // x5 = g +
add x6,x12,x13    // x6 = i + j
sub x20,x5,x6     // f = x5 -
x6
```

- Assignment

- Arg
- f in

• What if x5, x6, x20 were being used by the Calling procedure before calling leaf_example() ?

RISC-V Procedure Calling: Utilizing Stack Efficiently

- C code:

```
long long int leaf_example (long long int
g,
                                long long int
h,
                                long long
int i,
                                long long int
j) {
    long long int f;
    f = (g + h) - (i + j);
    return f;
}
```

- RISC-V code

- leaf_example:

```
add x5,x10,x11    // x5 = g +
add x6,x12,x13    // x6 = i + j
sub x20,x5,x6     // f = x5 -
```

x6

- Assignment

- Arg
- f in

• What if x5, x6, x20 were **NOT** being used by the Calling procedure before calling leaf_example() ?

RISC-V Procedure Calling: Utilizing Stack Efficiently

- To avoid saving and restoring a register whose value is never used, RISC-V introduces two groups of registers
- ***x5 – x7, x28 – x31: temporary registers***
 - *If used by the callee (called procedure), the callee does not preserve them*
- ***x8 – x9, x18 – x27: saved registers***
 - *If used by the callee (called procedure), the callee saves and restores them using Stack*

RISC-V Procedure Calling: Utilizing Stack Efficiently

- C code:

```
long long int leaf_example (long long int
g,
                                long long int
h,
                                long long
int i,
                                long long int
j) {
    long long int f;
    f = (g + h) - (i + j);
    return f;
}
```

- Assignment

- Arguments g, ..., j in x10, ..., x13
- f in x20, g+h and i+j in x5, x6
- Need to save x5, x6, x20 on stack

- RISC-V code

- leaf_example:

```
addi sp,sp,-24
sd x5,16(sp)
sd x6,8(sp)
sd x20,0(sp)
add x5,x10,x11    // x5 = g +
h
add x6,x12,x1      // x6 = i + j
sub x20,x5,x6      // f = x5 -
x6
addi x10,x20,0
ld x20,0(sp)
ld x6,8(sp)
ld x5,16(sp)
addi sp,sp,24
jalr x0,0(x1)
```

RISC-V Procedure Calling: Utilizing Stack

- C code:

```
long long int leaf_example (long long int
g,
                                long long int
h,
                                long long
int i,
                                long long int
j) {
    long long int f;
    f = (g + h) - (i + j);
    return f;
}
```

- RISC-V code

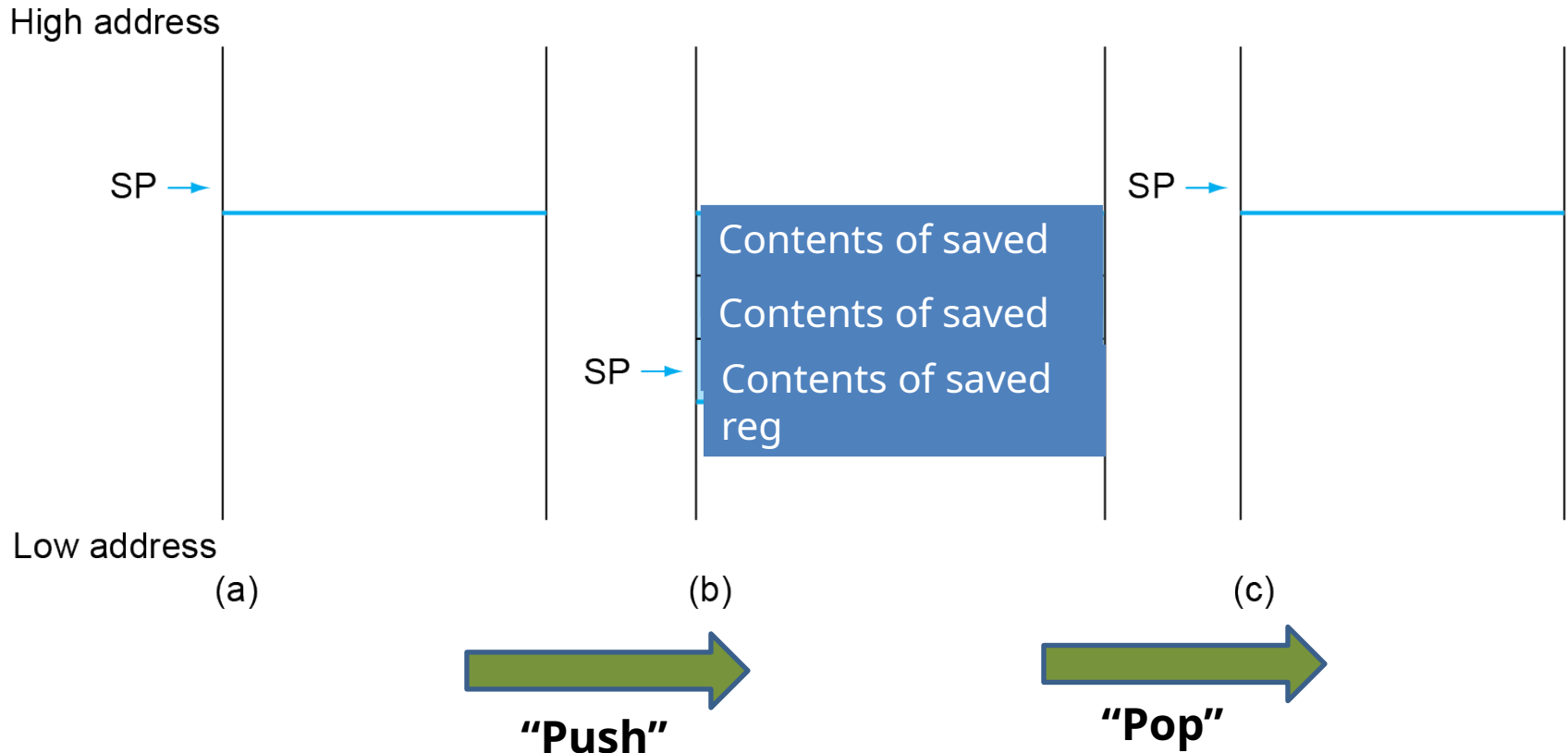
- leaf_example:

```
addi sp,sp,-8
sd x20,0(sp)
add x5,x10,x11    // x5 = g +
h
add x6,x12,x1      // x6 = i + j
sub x20,x5,x6      // f = x5 -
x6
addi x10,x20,0
ld x20,0(sp)
addi sp,sp,8
jalr x0,0(x1)
```

- Assignment

- Arguments g, ..., j in x10, ..., x13
 - f in x20, g+h and i+j in x5, x6
 - Need to save x20 on stack

RISC-V Procedure Calling: Utilizing Stack



RISC-V Procedure Calling: Utilizing Stack Efficiently

- To avoid saving and restoring a register whose value is never used, RISC-V introduces two groups of registers
- ***x5 – x7, x28 – x31: temporary registers***
 - *If used by the callee (called procedure), the callee does not preserve them*
- ***x8 – x9, x18 – x27: saved registers***
 - *If used by the callee (called procedure), the callee saves and restores them using Stack*

RISC-V: Procedure Calling (Summary v2)

- *The Caller (The calling procedure)*
 - *Pushes to stack any temporary registers used after the return from the procedure call*
 - *puts the parameter values in x10–x17*
 - *uses jal x1, X to branch to procedure X (sometimes named the callee).*
 - *After return from the procedure call, pops from stack any temporary register that was saved on the stack before going on the procedure call*
- *The Callee (The called procedure)*
 - *Pushes any saved registers used by the callee to stack*
 - *performs the calculations,*
 - *places the results in the same parameter/argument registers (x10-x17),*
 - *Pops any saved registers used by the callee from the stack*
 - *returns control to the caller using jalr x0, 0(x1).*

RISC-V Procedure Calling: Non-Leaf Procedures

- Leaf Procedure: Procedure that does not call other procedures
- There are extra complications with a non-leaf procedure
 - Suppose that the main program calls procedure A with an argument of 3, by placing the value 3 into register x10 and then using `jal x1, A`. Then suppose that procedure A calls procedure B via `jal x1, B` with an argument of 7, also placed in x10.
 - Since A hasn't finished its task yet, there is a conflict over the use of register x10.
 - Similarly, there is a conflict over the return address in register x1, since it now has the return address for B.

RISC-V Procedure Calling: Non-Leaf Procedures

- Leaf Procedure: Procedure that does not call other procedures
- There are extra complications with a non-leaf procedure
 - Suppose that the main program calls procedure A with an argument of 3, by placing the value 3 into register x10 and then using `jal x1, A`. Then suppose that procedure A calls procedure B via `jal x1, B` with an argument of 7, also placed in x10.
 - Since A hasn't finished its task yet, there is a conflict over the use of register x10.
 - Similarly, there is a conflict over the return address in register x1, since it now has the return address for B.
- Solution:
 - The caller pushes any argument registers (x10–x17) that are needed after the call.
 - The callee pushes the return address register x1.

RISC-V: Procedure Calling (Summary v3)

- *The Caller (The calling procedure)*
 - Pushes to stack *any argument registers (x10-17)* or temporary registers used after the return from the procedure call
 - puts the parameter values in x10-x17
 - uses *jal x1, X* to branch to procedure X (sometimes named the callee).
 - After return from the procedure call, pops from stack *any argument register* or temporary register that was saved on the stack before going on the procedure call
- *The Callee (The called procedure)*
 - *Pushes the return address (x1) to stack, if it is a non-leaf procedure*
 - Pushes any saved registers used by the callee to stack
 - performs the calculations *and make any procedure calls (if it is a non-leaf procedure)*
 - places the results in the same parameter/argument registers (x10-x17)
 - Pops any saved registers used by the callee from the stack
 - *Pops the return address from stack to x1, if it is a non-leaf procedure*
 - returns control to the caller using *jalr x0, 0(x1)*.

RISC-V: Procedure Calling (Final Summary)

- *The Caller (The calling procedure)*
 - *Pushes to stack any argument registers (x10-17) or temporary registers used after the return from the procedure call*
 - *puts the parameter values in x10-x17*
 - *uses jal x1, X to branch to procedure X (sometimes named the callee).*
 - *After return from the procedure call, pops from stack any argument register or temporary register that was saved on the stack before going on the procedure call*
- *The Callee (The called procedure)*
 - *Pushes the return address (x1) to stack, if it is a non-leaf procedure*
 - *Pushes any saved registers used by the callee to stack*
 - *performs the calculations and make any procedure calls (if it is a non-leaf procedure)*
 - *places the results in the same parameter/argument registers (x10-x17)*
 - *Pops any saved registers used by the callee from the stack*
 - *Pops the return address from stack to x1, if it is a non-leaf procedure*
 - *returns control to the caller using jalr x0, 0(x1).*

RISC-V Procedure Calling: Utilizing Stack

- C code:


```
long long int proc_A (long long int a) {
    long long int f;
    f = a + 1;
    g = proc_B(f)
    return g+a;
}

long long int proc_B (long long int b) {
    long long int f;
    f = b + 1;
    return f;
}
```
- RISC-V code
 - proc_A:
- RISC-V code
 - proc_B:

```
addi x20,x10,1
addi x10,x20,0

jalr x0,0(x1)
```
- Assignment
 - Arguments a and b in x10
 - f in x20, g in x 21

RISC-V Procedure Calling: Utilizing Stack

- C code:

```
long long int proc_A (long long int a) {  
    long long int f;  
    f = a + 1;  
    g = proc_B(f)  
    return g+a;  
}  
  
long long int proc_B (long long int b) {  
    long long int f;  
    f = b + 1;  
    return f;  
}
```

- RISC-V code

- proc_A:

- RISC-V code

- proc_B:

```
addi sp,sp,-8  
sd x20,0(sp)  
addi x20,x10,1  
addi x10,x20,0  
ld x20,0(sp)  
addi sp,sp,8  
jalr x0,0(x1)
```

- Assignment

- Arguments a and b in x10
 - f in x20, g in x 21

RISC-V Procedure Calling: Utilizing Stack

- C code:

```
long long int proc_A (long long int a) {
    long long int f;
    f = a + 1;
    g = proc_B(f)
    return g+a;
}
long long int proc_B (long long int b) {
    long long int f;
    f = b + 1;
    return f;
}
```

- Assignment

- Arguments a and b in x10
- f in x20, g in x 21

- RISC-V code

– proc_A:

addi x20,x10,1

addi x10, x20, 0

jal x1, proc_B

addi x21, x10, 0

x21

- RISC-V code

– proc_B:

addi sp,sp,-8

sd x20,0(sp)

addi x20,x10,1

addi x10,x20,0

ld x20,0(sp)

addi sp,sp,8

jalr x0,0(x1)

RISC-V Procedure Calling: Utilizing Stack

- C code:

```
long long int proc_A (long long int a) {
    long long int f;
    f = a + 1;
    g = proc_B(f)
    return g+a;
}
long long int proc_B (long long int b) {
    long long int f;
    f = b + 1;
    return f;
}
```

- Assignment

- Arguments a and b in x10
- f in x20, g in x 21

- RISC-V code

– proc_A:

```
addi x20,x10,1
addi sp,sp,-8
sd x10,0(sp)
addi x10, x20, 0
```

```
jal x1, proc_B
addi x21, x10, 0
ld x10,0(sp)
addi sp,sp,8
add x10, x10,
```

x21

- RISC-V code

– proc_B:

```
addi sp,sp,-8
sd x20,0(sp)
addi x20,x10,1
addi x10,x20,0
ld x20,0(sp)
addi sp,sp,8
jalr x0,0(x1)
```

RISC-V Procedure Calling: Utilizing Stack

- C code:

```
long long int proc_A (long long int a) {  
    long long int f;  
    f = a + 1;  
    g = proc_B(f)  
    return g+a;  
}  
  
long long int proc_B (long long int b) {  
    long long int f;  
    f = b + 1;  
    return f;  
}
```

- Assignment

- Arguments a and b in x10
- f in x20, g in x 21

- RISC-V code

- proc_A:

```
        addi sp,sp,-24  
        sd  x1,16(sp)  
        sd  x20,8(sp)  
        sd x21,0(sp)  
        addi x20,x10,1  
        addi sp,sp,-8  
        sd  x10,0(sp)  
        addi x10, x20, 0
```

```
        jal x1, proc_B  
        addi x21, x10, 0  
        ld  x10,0(sp)  
        addi sp,sp,8  
        add x10, x10, x21  
        ld  x21,0(sp)  
        ld  x20,8(sp)  
        ld  x1,16(sp)  
        addi sp,sp,24  
        jalr x0,0(x1)
```

- RISC-V code

- proc_B:

```
        addi sp,sp,-8  
        sd  x20,0(sp)  
        addi x20,x10,1  
        addi x10,x20,0  
        ld  x20,0(sp)  
        addi sp,sp,8  
        jalr x0,0(x1)
```

Instruction Encoding

add x9, x20, x21

0	21	20	0	9	51
---	----	----	---	---	----

0000000	10101	10100	000	01001	0110011
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Instruction Encoding

`add x9, x20, x21`

0	21	20	0	9	51
---	----	----	---	---	----

0000000	10101	10100	000	01001	0110011
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

- Instructions are encoded in binary
 - Called machine code
- RISC-V instructions
 - Encoded as 32-bit instruction words

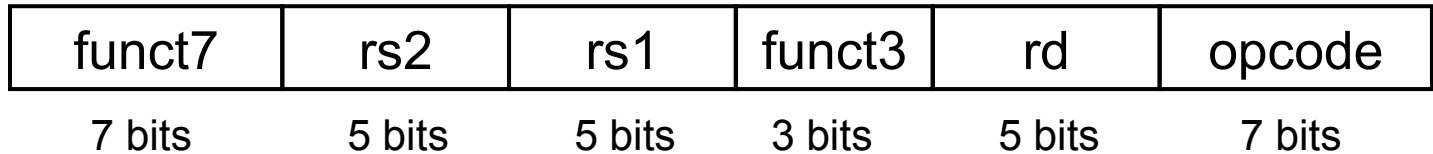
Hexadecimal: Review

- Base 16
 - Compact representation of bit strings
 - 4 bits per hex digit

0	0000	4	0100	8	1000	c	1100
1	0001	5	0101	9	1001	d	1101
2	0010	6	0110	a	1010	e	1110
3	0011	7	0111	b	1011	f	1111

- Example: eca8 6420
 - 1110 1100 1010 1000 0110 0100 0010 0000

RISC-V R-format Instructions



- Instruction fields
 - opcode: operation code
 - rd: destination register number
 - funct3: 3-bit function code (additional opcode)
 - rs1: the first source register number
 - rs2: the second source register number
 - funct7: 7-bit function code (additional opcode)

```
add x9, x20, x21
```

RISC-V I-format Instructions



- Instruction fields
 - opcode: operation code
 - rd: destination register number
 - funct3: 3-bit function code (additional opcode)
 - rs1: source or base address register number
 - Immediate: constant operand, or offset added to base address (2's complement, sign extended)

```
ld x9, 64(x22)
```


RISC-V S-format Instructions

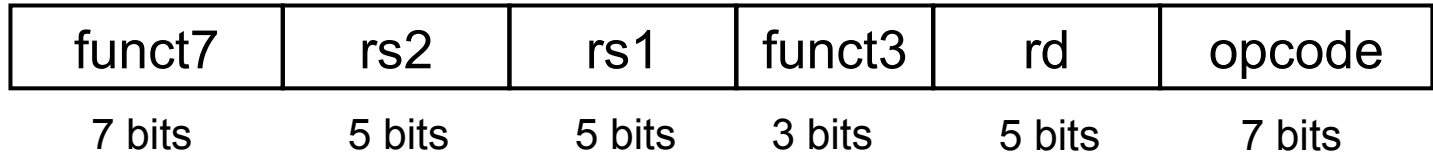


- Instruction fields
 - opcode: operation code
 - funct3: 3-bit function code (additional opcode)
 - rs1: base address register number
 - rs2: source operand register number
 - immediate: offset added to base address (Split so that rs1 and rs2 fields always
in the same place across various instruction formats)

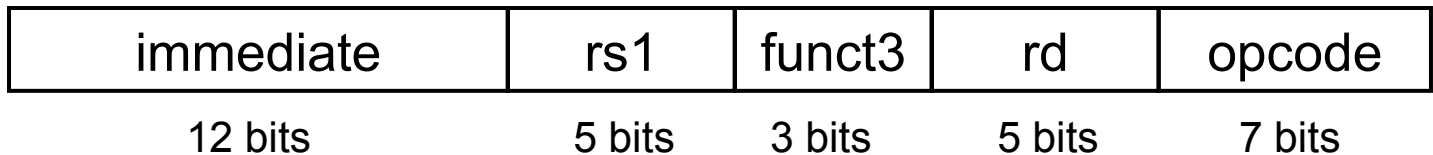
```
sd    x9, 240(x10)
```

RISC-V Instruction Encoding Format

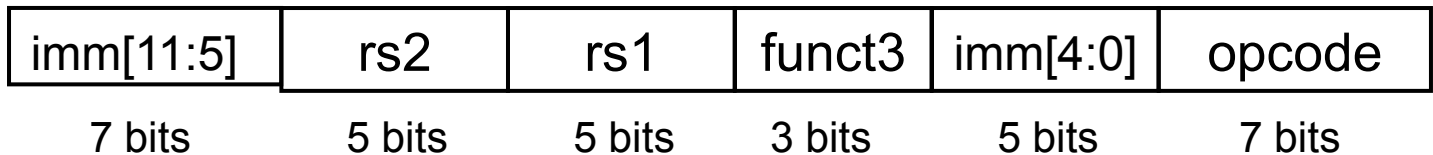
- R-format



- I-format



- S-format



RISC-V Instructions & their Encoding Format

Instruction	Format	funct7	rs2	rs1	funct3	rd	opcode
add (add)	R	0000000	reg	reg	000	reg	0110011
sub (sub)	R	0100000	reg	reg	000	reg	0110011
Instruction	Format	immediate		rs1	funct3	rd	opcode
addi (add immediate)	I	constant		reg	000	reg	0010011
ld (load doubleword)	I	address		reg	011	reg	0000011
Instruction	Format	immediate	rs2	rs1	funct3	immediate	opcode
sd (store doubleword)	S	address	reg	reg	011	address	0100011

Instruction Encoding Example

`A[30] = h + A[30] + 1;`

is compiled into

```
ld    x9, 240(x10)    // Temporary reg x9 gets A[30]
add   x9, x21, x9     // Temporary reg x9 gets h+A[30]
addi  x9, x9, 1       // Temporary reg x9 gets h+A[30]+1
sd    x9, 240(x10)    // Stores h+A[30]+1 back into A[30]
```

Instruction Encoding Example

`A[30] = h + A[30] + 1;`

is compiled into

```
ld    x9, 240(x10)    // Temporary reg x9 gets A[30]
add   x9, x21, x9      // Temporary reg x9 gets h+A[30]
addi  x9, x9, 1        // Temporary reg x9 gets h+A[30]+1
sd    x9, 240(x10)    // Stores h+A[30]+1 back into A[30]
```

immediate	rs1	funct3	rd	opcode
240	10	3	9	3

funct7	rs2	rs1	funct3	rd	opcode
0	9	21	0	9	51

immediate	rs1	funct3	rd	opcode
1	9	0	9	19

immediate[11:5]	rs2	rs1	funct3	immediate[4:0]	opcode
7	9	10	3	16	35

Instruction Encoding Example

`A[30] = h + A[30] + 1;`

is compiled into

```
ld    x9, 240(x10)    // Temporary reg x9 gets A[30]
add   x9, x21, x9      // Temporary reg x9 gets h+A[30]
addi  x9, x9, 1        // Temporary reg x9 gets h+A[30]+1
sd    x9, 240(x10)    // Stores h+A[30]+1 back into A[30]
```

immediate	rs1	funct3	rd	opcode
000011110000	01010	011	01001	0000011

funct7	rs2	rs1	funct3	rd	opcode
0000000	01001	10101	000	01001	0110011

immediate	rs1	funct3	rd	opcode
000000000001	01001	000	01001	0010011

immediate[11:5]	rs2	rs1	funct3	immediate[4:0]	opcode
0000111	01001	01010	011	10000	0100011